

How Far Is This Project? — Plain-Language Status Report

This document explains, in simple terms, what is actually in this repository, what works, what doesn't (or isn't real), how the pieces fit together, and what every single file does. It's written for someone who wants to understand the project without reading all the code first.

1. The one-paragraph summary

This is a working demo (not a finished product) of a website where households can sell and buy electricity from each other. Some households produce more power than they use (solar panels, wind) and can sell the extra. Others use more than they produce and need to buy some. A fake "day" runs automatically in the background (sun goes up, sun goes down) so production numbers change over time like a real solar panel would. When someone buys electricity, a digital coin (called TEC) moves from the buyer to the seller, and that transaction gets permanently recorded on a simple blockchain built into this app. Everything runs on your own computer — no real money, no real electricity, no real blockchain network is required to try it, although a real one (Hedera) can be plugged in if you want.

2. What works right now (tested and verified)

These things were actually run and checked — not just written and assumed to work:

- **Creating an account / logging in** — works. You pick a household type (producer, consumer, or prosumer = both) and get a password-protected account.
- **Submitting an energy reading** — works. You type in how much power you produced and consumed; the app calculates whether you have extra (surplus) or a shortage (deficit).
- **Automatic simulation** — works. Every few seconds, all households get a new automatic reading following a realistic “sun goes up at 6am, peaks at noon, down by 6pm” curve, without you doing anything.
- **Auto-conversion of surplus into coins** — works. If you produced more than you used, the extra amount is automatically turned into TEC coins in your wallet.
- **Creating a sell offer (marketplace listing)** — works. You can’t offer to sell more than you actually have banked up.
- **Buying energy from someone’s offer** — works. Coins move from buyer to seller, the offer’s remaining amount goes down, and it’s all recorded.
- **Blockchain recording** — works, but it’s a **simplified, locally-built blockchain** (explained in section 4), not a real public blockchain by default.
- **Preventing cheating** — tested and confirmed:
 - Can’t sell more energy than you have.
 - Can’t buy with coins you don’t have.
 - Can’t buy more than an offer has left.
 - Can’t accidentally “execute” the same purchase twice (double-spending).
 - Two people can’t both buy the last bit of the same offer at the same time.
- **Dashboard, wallet, transaction history, blockchain explorer, microgrid map** — all built and confirmed to show live data on-screen (checked with real screenshots in a browser, not just “it should work”).
- **Automated tests** — 30 automated checks exist and all 30 currently pass. They cover the money-and-energy math, the offer/purchase rules, and the blockchain rules.

3. What does NOT work / is NOT real

Being direct about the limitations:

- **The blockchain is not a real public blockchain by default.** It's a simplified version built from scratch inside this app (chained, tamper-evident blocks with hashes) — it demonstrates the *idea* correctly, but it is not decentralized and nobody outside this app can verify it independently. Turning on real Hedera Hashgraph mode is possible, but requires YOU to create a free account and enter your own keys — this was never turned on or tested with real credentials, because no real account was ever provided.
- **No real electricity or real meters are involved.** All production/consumption numbers are either typed in by hand or generated by the built-in simulator. There is no connection to a real solar panel, smart meter, or power grid.
- **No real money.** TEC coins only exist inside this app's own database.
- **The "smart contract" is regular code, not a blockchain smart contract.** The rules (can't oversell, can't overspend, etc.) are enforced by the backend server, the same way a normal website enforces rules — not by code running on a blockchain that nobody can secretly change.
- **Docker was written but never fully test-run in this environment,** because Docker's background engine wasn't switched on when this was built. The configuration file was checked for syntax errors (that passed), but nobody has actually watched `docker compose up` finish successfully end-to-end. Running it locally without Docker *was* fully tested and works.
- **No real login security hardening.** Login uses a standard method (JWT), fine for a demo, but the default secret keys in the example settings file are placeholders — they must be changed before anyone would consider this "production-ready."
- **No email, phone verification, or password reset.** If you forget your password there is no recovery flow.
- **Not mobile-app tested.** Only tested as a website in a desktop browser window.

4. Architecture — how the pieces talk to each other

Think of it as four layers, stacked:

```
[ Browser / Website ]      <- what you see and click
    | (talks over the internet using simple web requests)
[ Backend Server ]        <- the "brain": checks rules, does the math
    |
    |-- [ Database file ]  <- remembers households, offers, trades, coins
    |
    |-- [ Blockchain part ] <- separately records every coin movement
```

- The **website** (frontend) never trusts itself — it just displays data and sends requests. All the real rule-checking happens on the **server** (backend), so a user can't cheat by tampering with their own browser.
- The server is split into small specialized pieces (one for households, one for offers, one for trades, one for coins, one for the blockchain) instead of one giant file — this makes it easier to find and fix things.
- The blockchain part is deliberately kept separate and swappable: the rest of the app doesn't care whether it's talking to the simplified built-in blockchain or a real one — it just asks "record this transaction" and gets back a confirmation, the same way either way.
- The database only stores facts (who owns what, what was offered), never rules — rules live in the server code.

The trading flow, step by step

```

Producer's panel makes power
-> reading gets submitted (by hand or by the simulator)
-> app calculates: production - consumption = surplus or deficit
-> if surplus: coins are automatically created and given to the producer
-> producer lists some of their surplus for sale (an "offer")
-> consumer sees the offer, picks how much to buy
-> app double-checks the offer still has enough left, and the buyer has enough coins
-> coins move from buyer to seller
-> transaction is written into the blockchain part
-> offer is updated (or closed if fully bought)
-> everyone can see the result in their history / the blockchain explorer page

```

5. Role of every file

Root folder

File	What it's for
README.md	Main project overview, setup instructions, feature list.
how-far.md	This document — plain-language status report.
BLOCKCHAIN_ENERGY_TRADING_DEEP_DIVE.md	The original reference document this project was inspired by/built on (describes a similar real system using Hedera).
docker-compose.yml	One file that can start both the server and the website together in containers, with one command.
.gitignore	Tells version control which files to ignore (generated files, secrets, dependencies).
docs/ARCHITECTURE.md	Diagrams and deeper explanation of how the layers connect.
docs/API.md	List of every web address (endpoint) the server understands, and what each one does.

Backend (`backend/` — the server, written in TypeScript/Node.js)

Top level

File	What it's for
<code>package.json</code>	Lists what software libraries the server needs, and the shortcut commands (<code>npm run dev</code> , <code>npm test</code> , etc.).
<code>tsconfig.json</code>	Settings for the TypeScript language checker.
<code>vitest.config.ts</code>	Settings for the automated test runner.
<code>.env.example</code>	A template of configuration settings (ports, secret keys, optional real-blockchain settings) — copy it to <code>.env</code> to customize.
<code>.dockerignore</code>	Tells Docker which files to skip when packaging the server.
<code>Dockerfile</code>	Instructions for building the server into a Docker container.

`src/domain/` — the shared vocabulary

File	What it's for
<code>types.ts</code>	Defines what a "Household", "Offer", "Trade", "Token Transaction", etc. look like — the shared nouns every other file uses.

`src/db/` — the memory of the app

File	What it's for
<code>schema.sql</code>	The blueprint for the database: what tables exist and what columns they have.
<code>database.ts</code>	Opens the database file, creates it if missing, hands out a connection.
<code>repositories/householdRepository.ts</code>	All the ways to save/read/update a household in the database.
<code>repositories/measurementRepository.ts</code>	Saves and reads energy readings.
<code>repositories/offerRepository.ts</code>	Saves and reads marketplace offers.
<code>repositories/tradeRepository.ts</code>	Saves and reads completed/pending purchases.
<code>repositories/tokenTransactionRepository.ts</code>	Saves and reads every coin movement.
<code>repositories/blockchainRepository.ts</code>	Saves and reads blockchain blocks and blockchain transactions.

`src/blockchain/` — the ledger

File	What it's for
<code>BlockchainService.ts</code>	The "contract" that any blockchain implementation must follow (so they're interchangeable).
<code>hash.ts</code>	The math that creates a tamper-evident fingerprint (hash) for each block.
<code>LocalBlockchainService.ts</code>	The default, simplified, built-in blockchain — no internet or real crypto account needed.
<code>HederaBlockchainService.ts</code>	The real-blockchain version — only activates if you provide real Hedera account credentials.
<code>index.ts</code>	Decides automatically which of the two above to use, based on your settings.

src/services/ — the actual business rules ("the brain")

File	What it's for
<code>authService.ts</code>	Handles registering and logging in households; creates login tokens.
<code>householdService.ts</code>	Fetches household info and their history.
<code>measurementService.ts</code>	Records energy readings and calculates surplus/deficit; triggers coin creation.
<code>tokenService.ts</code>	Moves coins between households; keeps balances accurate and prevents overspending.
<code>marketplaceService.ts</code>	Creates and cancels sell offers; makes sure you can't offer more than you have.
<code>tradeService.ts</code>	The purchase process: checks everything is valid, moves coins, records it on the blockchain, updates the offer — all safely and without allowing cheating.
<code>analyticsService.ts</code>	Adds up totals for the Dashboard and Microgrid pages (total production, total trades, etc.).

src/simulation/ — the fake "sun and daily life"

File	What it's for
<code>curves.ts</code>	The math formulas for "how much sun is there at this hour" and "how much power does a house use at this hour."
<code>simulationService.ts</code>	Runs on a timer in the background, feeding new fake readings into the system automatically.

src/api/ — the front door (what the website talks to)

File	What it's for
<code>index.ts</code>	Connects all the route files together under one address.
<code>routes/auth.routes.ts</code>	Web addresses for register/login.
<code>routes/households.routes.ts</code>	Web addresses for viewing households.
<code>routes/energy.routes.ts</code>	Web addresses for submitting/viewing energy readings.
<code>routes/market.routes.ts</code>	Web addresses for creating offers, cancelling, and buying.
<code>routes/trades.routes.ts</code>	Web addresses for viewing past trades.
<code>routes/tokens.routes.ts</code>	Web addresses for checking coin balances and history.
<code>routes/transactions.routes.ts</code>	Web address for the full, site-wide transaction feed.
<code>routes/blockchain.routes.ts</code>	Web addresses for the blockchain explorer (blocks, transactions, status).
<code>routes/dashboard.routes.ts</code>	Web addresses for the Dashboard and Microgrid summary numbers.

`src/middleware/` — the security guards

File	What it's for
<code>auth.ts</code>	Checks that a request has a valid login token before allowing an action.
<code>errorHandler.ts</code>	Catches errors anywhere in the app and turns them into a clean, safe error message instead of crashing.

`src/ws/` — instant notifications

File	What it's for
<code>notificationHub.ts</code>	Sends a live pop-up-style message to a seller the instant someone buys their offer (without needing to refresh the page).

`src/utils/` — small helpers

File	What it's for
<code>errors.ts</code>	Defines the standard error types used everywhere (e.g. "not found," "not allowed").
<code>crypto.ts</code>	Encrypts/decrypts sensitive data (like a real blockchain private key) before saving it.

`src/scripts/` — one-time or manual jobs

File	What it's for
<code>seedData.ts</code>	The actual list of 10 demo households and their starting numbers.
<code>seed.ts</code>	A command you can run manually to add the demo households.
<code>initToken.ts</code>	A one-time setup script only needed if you switch on the real Hedera blockchain — creates the real TEC coin.

`src/config/` and root files

File	What it's for
<code>config/env.ts</code>	Reads all the settings (ports, secret keys, simulation speed, blockchain mode) from the <code>.env</code> file.
<code>container.ts</code>	Builds one instance of every service and wires them together (so the rest of the app doesn't have to).
<code>app.ts</code>	Sets up the web server itself (security headers, rate limiting, connects all the routes).
<code>server.ts</code>	The actual starting point — boots the database, seeds demo data if empty, starts the web server, starts the background simulation.

`tests/` — the automated checks

File	What it's for
<code>testContainer.ts</code>	Helper to quickly set up a fresh, empty test version of the app for each test.
<code>unit/measurement.test.ts</code>	Checks surplus/deficit math is correct.
<code>unit/marketplace.test.ts</code>	Checks offer creation/cancelling rules.
<code>unit/trade.test.ts</code>	Checks the full purchase process, including all the cheating-prevention rules.
<code>unit/token.test.ts</code>	Checks coin transfers and balance rules.
<code>unit/blockchain.test.ts</code>	Checks blocks are created correctly and tampering is detected.
<code>integration/api.test.ts</code>	Checks the actual web addresses work correctly from outside, like a real user would call them.

Frontend (`frontend/` — the website, written in React)

Top level

File	What it's for
<code>package.json</code>	Lists what libraries the website needs and shortcut commands.
<code>tsconfig.json</code>	TypeScript settings for the website code.
<code>vite.config.ts</code>	Settings for the tool that runs/builds the website; also forwards API requests to the backend during development.
<code>index.html</code>	The single, near-empty webpage that the whole website loads into.
<code>nginx.conf</code>	Settings for serving the finished website in Docker.
<code>Dockerfile</code> / <code>.dockerignore</code>	Instructions to package the website into a Docker container.

`src/` top level

File	What it's for
<code>main.tsx</code>	The very first code that runs — starts the website and wraps it with login-state tracking.
<code>App.tsx</code>	Defines which page shows for which web address (Dashboard, Marketplace, etc.) and blocks pages behind login.
<code>index.css</code>	All the visual styling (colors, spacing, buttons, tables) for the whole site.
<code>types.ts</code>	The shared shapes of data (matching the backend's) so the website code knows what a "Household" or "Offer" looks like.

`src/api/`

File	What it's for
<code>client.ts</code>	The one place that actually sends requests to the backend server and handles login tokens/errors.

`src/context/`

File	What it's for
<code>AuthContext.tsx</code>	Remembers who's logged in across the whole site; provides login/register/logout everywhere.

`src/hooks/`

File	What it's for
<code>usePolling.ts</code>	A reusable helper that automatically re-fetches data every few seconds so pages stay live/up to date.

`src/components/` — reusable building blocks

File	What it's for
Layout.tsx	The sidebar menu and overall page frame shown on every screen.
Badge.tsx	The small colored pill labels (e.g. "Producer", "Completed").
StatCard.tsx	The little number-in-a-box widgets used on the Dashboard.

src/pages/ — one file per screen

File	What it's for
LoginPage.tsx	Log in or register a new household.
DashboardPage.tsx	Overview numbers, charts, recent trades, blockchain summary.
HouseholdsPage.tsx	List of every household and their live numbers.
EnergyMonitoringPage.tsx	Your own production/consumption history chart, and a form to submit a reading by hand.
MarketplacePage.tsx	Browse offers, list your own surplus for sale, buy from others.
MyOffersPage.tsx	Offers you've listed, with a cancel button.
MyPurchasesPage.tsx	Energy you've bought (and sold, if applicable).
WalletPage.tsx	Your coin balance and coin transaction history.
TransactionsPage.tsx	Every coin transaction across the whole system.
BlockchainExplorerPage.tsx	Browse blocks and see what transactions are inside each one.
MicrogridPage.tsx	Visual grouping of producers vs. consumers and their net energy flow.

6. How to run it

Easiest way — Docker (one command, if Docker Desktop is running)

```
docker compose up --build
```

Then open the website at <http://localhost:8080>. The server runs at <http://localhost:4000> in the background. The database starts empty, so the app automatically creates 10 demo households the first time it boots — you don't need to do anything extra.

Note: this exact command was written and its configuration file was checked for mistakes, but it was not personally watched running to completion in this environment because Docker's background engine wasn't turned on at the time. It should work, but this is the one thing that hasn't been visually confirmed.

Manual way — two terminals (fully tested, confirmed working)

Terminal 1 — start the server:

```
cd backend
npm install
npm run dev
```

Leave this running. It listens on `http://localhost:4000` and creates the demo households automatically the first time.

Terminal 2 — start the website:

```
cd frontend
npm install
npm run dev
```

Leave this running too. Open <http://localhost:5173> in your browser.

Logging in

Use any of these demo accounts (all use the same password):

- Producers: `producer-1` , `producer-2` , `producer-3` , `producer-4` , `producer-5`
- Consumers: `consumer-1` , `consumer-2` , `consumer-3` , `consumer-4` , `consumer-5`
- Password (for all of them): `password123`

Running the automated tests

```
cd backend
npm test
```

You should see `30 passed` .

Turning on the real blockchain (optional, advanced, not required)

Only do this if you want real blockchain transactions instead of the built-in simplified one:

1. Make a free account at `portal.hedera.com` .
2. Put your account ID and private key into `backend/.env` (copy `backend/.env.example` first).
3. Run `npm run init:token` once inside `backend/` — this creates the real coin and prints an ID.
4. Put that ID into `.env` too, and restart the server.

If you skip all of this (the default), everything still works, just with the built-in simplified blockchain instead of a real one.